



Universidad
Indoamérica

ユニット 8・オペレーティングシステムの基礎

ネットワーク：ソケット

Universidad Tecnológica Indoamérica

演習 TD7

2つの接続モード

接続型・SOCK_STREAM (TCP)

- 接続は一度だけ確立し、その後データをやり取りする
- **電話**のように
- パイプのような信頼できる
バイトストリーム：誤りを検出し、再送し、順序を保つ

非接続型・SOCK_DGRAM (UDP)

- 送信のたびに独立したメッセージ
(データグラム)
- **電報**のように
- 保証なし：消失、重複、順序の入れ替わり

ストリーム対データグラムを測る

実際の出力

```
$ ./udp
TCP: 3 writes (one, two, three) → read #1 returns 11 bytes: "onetwothree"
UDP: 3 sendto (one, two, three) → recvfrom #1: 3 "one", recvfrom #2: 3 "two", recvfrom #3: 5
"three"
UDP: receive buffer 8192 bytes, 10000 sendto of 100 bytes succeeded, 8 received, 9992 lost
kernel counter Udp RcvbufErrors grew by 9992
```

- TCP は区切りを保たない：各メッセージの終わりは**アプリケーションのプロトコル**が示す（改行、長さ...）
- UDP は同じマシンの上でさえ、**誰にも知らせずに**データグラムを失う

```
int socket(int domain, int type, int protocol);
```

domain	AF_UNIX : ローカル、アドレスはパス・ AF_INET : IPv4・ AF_INET6 : IPv6
type	SOCK_STREAM : 接続型・ SOCK_DGRAM : 非接続型・ SOCK_RAW : 生のパケット (CAP_NET_RAW が必要)
+ フラグ	SOCK_NONBLOCK、 SOCK_CLOEXEC (Linux 2.6.27 以降) : type に で組み合わせる
protocol	0 : システムが選ぶ・ IPPROTO_TCP・ IPPROTO_UDP

修正 (スライド 182) : 「AF_UNIX : ローカル (127.0.0.1)」は 2 つのことを混ぜている。127.0.0.1 はループバックインターフェース上の AF_INET で、AF_UNIX は IP を使わない。

ローカルドメイン：ソケット型のファイル

実際の出力

```
$ ./local fso.sock  
bind created fso.sock: a file of type socket (the 's' of ls -l), sun_path holds 108 bytes  
the server asks the kernel who connected: pid = mine, uid 1000 (my user)  
read 5 bytes; after close the file stays until unlink  
still there after close: yes
```

- `bind` はファイルシステムに項目を作る（`ls -l` の `s`）。`unlink` まで残る
- カーネルは**誰が**接続したかをサーバに教える（`SO_PEERCRED`）：`systemd`、`D-Bus`、`/var/run/docker.sock` はこれで権限を決める

インターネットドメイン：アドレス + ポート

- ポート（16 ビット）はマシン内のサービスを識別する
- 1024 未満のポート：特権のみ
（`CAP_NET_BIND_SERVICE`）。Linux ではこの境界は `net.ipv4.ip_unprivileged_port_start`
- サービス名： `/etc/services`

実際の出力

```
$ ./addr 7000
...
ip_unprivileged_port_start = 1024
sin_port = htons(80):      bind: Permission denied
...
```

サービス	ポート	現在
ftp	21/tcp	暗号化なし。sftp か https を使う
telnet	23/tcp	暗号化なし。ssh (22/tcp) に置き換わった
http	80/tcp	ほぼ常に https にリダイレクトする
https	443/tcp, 443/udp	TLS。HTTP/3 は UDP (QUIC) の上で動く

bind : アドレス + ポート

C

```
int bind(int sock, const struct sockaddr *addr, socklen_t len)
struct sockaddr { sa_family_t sa_family; char sa_data[14]; }
struct sockaddr_un {          /* AF_UNIX (= AF_LOCAL) */
    sa_family_t sun_family;
    char        sun_path[108]; /* a path */
};
struct sockaddr_in {          /* AF_INET */
    sa_family_t  sin_family;
    in_port_t    sin_port;    /* network order */
    struct in_addr sin_addr;   /* 4 bytes */
    unsigned char sin_zero[8];
};
/* AF_INET6: struct sockaddr_in6 (16-byte address) */
```

- ドメインごとに構造体がある。どれもサイズとともに `struct sockaddr *` として渡す
- `struct sockaddr_storage` はそのどれでも入れられる（たとえば `accept` で）

実際の出力

```
$ ./addr 7000
...
sizeof: sockaddr 16, sockaddr_in 16, sockaddr_in6 28,
sockaddr_un 110, sockaddr_storage 128
```

スライド 187 : htons が抜けていた

C

```
/* slide 187 */  
s.sin_port = 7000;           /* wrong */  
/* fixed (sock.c) */  
s.sin_port = htons(7000);    /* network order */  
s.sin_addr.s_addr = htonl(INADDR_ANY);
```

- ネットワークはビッグエンディアン、x86-64 と ARM64 はリトルエンディアン (TDI で od を使って見た)

0x1B58 を逆向きに読むと 0x581B = **22555** : サーバは別のポートで待ち受けていて、誰も見つけられない。

実際の出力

```
$ ./addr 7000  
port 7000 = 0x1b58; htons(7000) = 0x581b in memory order  
sin_port = 7000:           the kernel bound port 22555  
sin_port = htons(7000):    the kernel bound port 7000  
...
```

gethostbyname から getaddrinfo へ

C tds/td7/addr.c

```
const char *names[][2] = { { "localhost", "http" }, { "
for (int k = 0; k < 2; k++) {
    struct addrinfo hints = { .ai_family = AF_UNSPEC, .
    int e = getaddrinfo(names[k][0], names[k][1], &hints);
    exit_if(e != 0, gai_strerror(e));
    for (struct addrinfo *p = res; p != NULL; p = p->ai
...
    freeaddrinfo(res);
}
```

実際の出力

```
$ ./addr 7000
...
getaddrinfo("localhost", "http"): IPv4 127.0.0.1 port 80
getaddrinfo("::1", "https"): IPv6 ::1 port 443
...
```

- **gethostbyname** (スライド 187) : IPv4 のみ、スレッドセーフでない、2008 年から POSIX の外
- **getaddrinfo** (RFC 3493) : 名前 + サービス → **connect** にそのまま使える IPv4 と IPv6 のアドレスの一覧

```
int listen(int sock, int backlog);
```

- サービスをシステムに宣言する：ソケットは**受動的**になり、接続を待つだけになる
- `backlog` : `accept` を待てる**確立済みの**接続の数（Linux は `backlog + 1` 個まで受け入れる）
- キューが満杯だと SYN は捨てられ、クライアントが再送する（1 秒、2 秒、4 秒...）

実際の出力

```
$ ./backlog 4 8
listen(4), 8 clients connect
0.3 s, no accept yet:      accept queue 5 | clients ESTABLISHED 5, SYN_SENT 3
accept 5 (total 5), wait:  accept queue 3 | clients ESTABLISHED 8, SYN_SENT 0
accept 3 (total 8), wait: all 8 clients accepted
```

accept : キューから接続を取り出す

```
C
int accept(int sock, struct sockaddr *a, socklen_t *len);  /* a new descriptor */
```

- `a` と `*len` にクライアントのアドレスが入る (不要なら `NULL`)
- キューに接続が来るまでブロックする
- そのクライアント用の**新しいディスクリプタ**を返す。待ち受け用のディスクリプタは受け付けを続ける

```
C tds/td7/sock.c
```

```
/* 5. accept blocks until a client connects and returns
int client = accept(s, NULL, NULL);
exit_if(client == -1, "accept");
printf("connection accepted, closing\n");
```

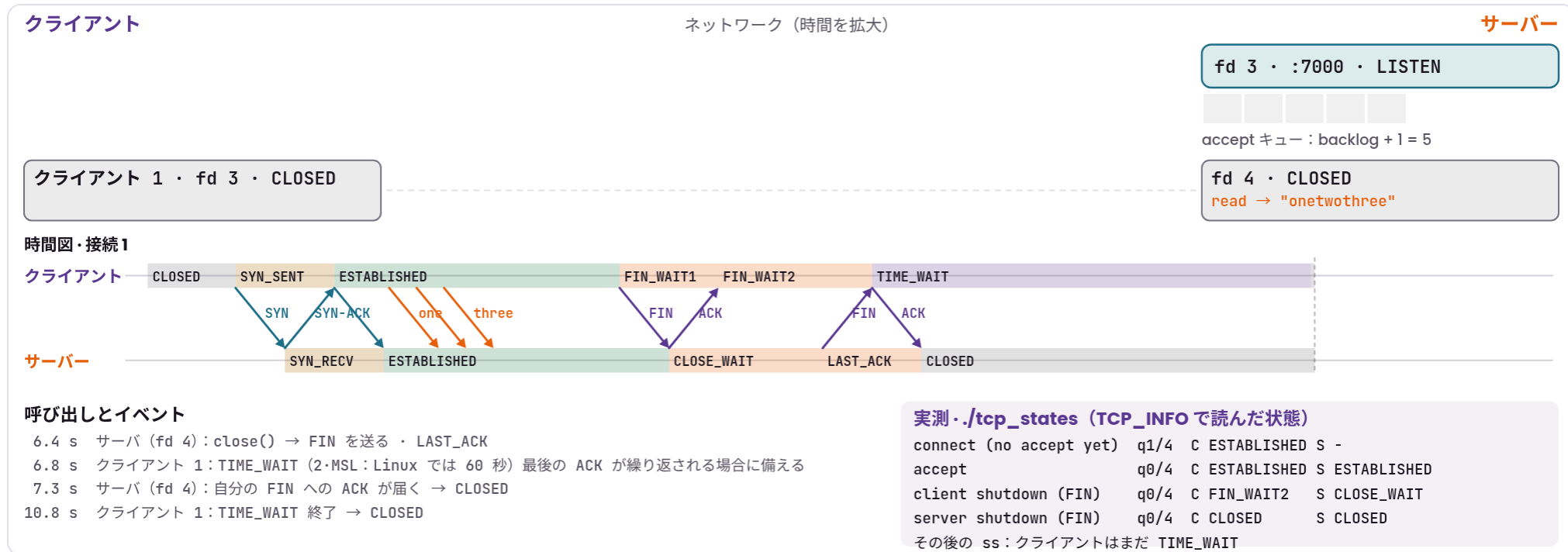
修正 (スライド 189) : 返すのは「`> 0`」ではなく **0 以上**のディスクリプタ (標準入力を閉じていれば 0 も有効)、エラーなら `-1`。 `accept4(..., SOCK_CLOEXEC)` は、ディスクリプタが `exec` に漏れるのを防ぐ。

connect : 接続を要求する

```
C
int connect(int sock, const struct sockaddr *srv, socklen_t len);
```

- サーバのアドレスへ接続を要求する：カーネルが **3 ウェイハンドシェイク** (SYN、SYN-ACK、ACK) を行う
- ハンドシェイクが終わるまでブロックする： **accept** は待たない
- よくあるエラー： **ECONNREFUSED** (そのポートで誰も待ち受けていない)、 **ETIMEDOUT**
- **SOCK_NONBLOCK** では **EINPROGRESS** を返し、ハンドシェイクは勝手に進む (**backlog.c** はこうして 8 つの接続を一度に開く)

接続をセグメントごとに



accept が返したディスクリプタで

- 書く： `write(fd, buf, n)` または `send(fd, buf, n, flags)`
- 読む： `read(fd, buf, n)` または `recv(fd, buf, n, flags)`
- 閉じる： `close(fd)`。 `shutdown(fd, SHUT_WR)` は書き込み方向だけを閉じる（FIN を送る）

ユニット 2 と 5 で学んだことがここでも当てはまる：

- 部分的な読み書き：ループする（`write_all`）
- `read` → 0：相手側が閉じた
- 閉じたソケットに書く → `SIGPIPE`。
`MSG_NOSIGNAL` か、シグナルを無視して避ける

状態を **TCP_INFO** でカーネルから読む

実際の出力

```
$ ./tcp_states PORT; ss -tn state time-wait '( dport = :PORT )'
socket+bind+listen(4)      | listener LISTEN (queue 0/4) | client -           | server -
connect (no accept yet)    | listener LISTEN (queue 1/4) | client ESTABLISHED | server -
accept                     | listener LISTEN (queue 0/4) | client ESTABLISHED | server ESTABLISHED
write/read                  | server read 6 bytes
client shutdown (FIN)      | listener LISTEN (queue 0/4) | client FIN_WAIT2   | server CLOSE_WAIT
server read                 | server read returns 0: end of file
server shutdown (FIN)      | listener LISTEN (queue 0/4) | client CLOSED      | server CLOSED
                           | the client's end moved to a TIME_WAIT entry of the kernel (60 s, see ss)
Recv-Q Send-Q Local Address:Port Peer Address:Port
0      0      127.0.0.1:42772    127.0.0.1:PORT
```

非接続モード： `sendto` と `recvfrom`

C

```
ssize_t sendto(int sock, const void *buf, size_t len, int flags,  
               const struct sockaddr *dest, socklen_t dlen);  
ssize_t recvfrom(int sock, void *buf, size_t len, int flags,  
                 struct sockaddr *src, socklen_t *slen);
```

- `sendto` = `connect` + `send` を1回の呼び出しで。 `recvfrom` = `accept` + `recv` で、送り手も教える
- 送受信したバイト数を返す（エラーなら -1）。 `recvfrom` 1回 = **データグラム1つ丸ごと**（バッファが小さければ残りは失われる）
- UDP ソケットにも `connect` できる：宛先を固定し、 `send` / `recv` が使えるようになる

誰が何を、どの順で呼ぶか

接続型 (TCP)	
サーバー	クライアント
socket · bind · listen	socket
accept (ブロック)	connect → ハンドシェイク
read ⇔ write	write ⇔ read
close	close

非接続型 (UDP)	
サーバー	クライアント
socket · bind	socket
recvfrom (ブロック)	sendto
sendto	recvfrom
close	close

よく知られたポートを bind するのはサーバだけ。クライアントには一時的なポートが自動で割り当てられる。

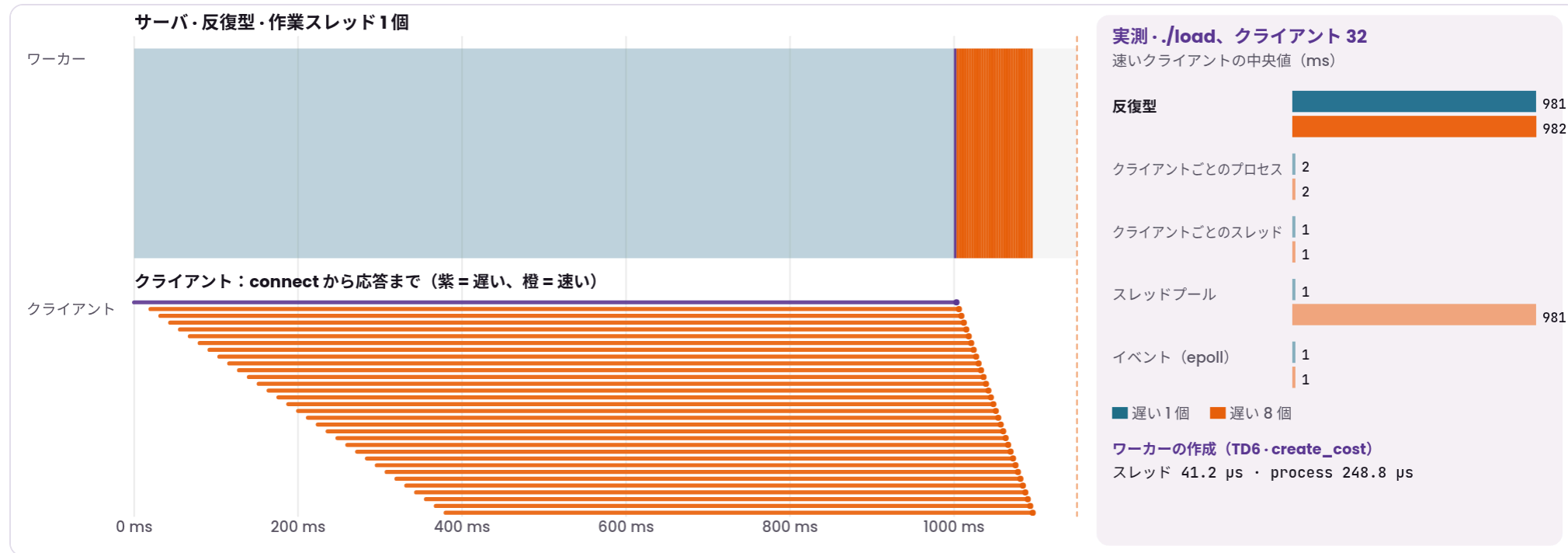
サーバーのモデル

- **反復型**：一度にクライアント 1 つ
- **マルチプロセスまたはマルチスレッド**：
メインスレッドが `accept` し、処理のための
プロセスかスレッドを作り、`accept` に戻る
- **スレッドプール**：同じだが、スレッドの数に上限が
ある。生産者はメインスレッド（接続）、消費者は
処理スレッド

現在はさらに

イベントループ：1 つのスレッドと `epoll`
(Linux、2002 年) で何千もの接続を
処理する。C10K 問題 (D. Kegel、1999 年) への
答えで、nginx や Node.js のモデル。 `io_uring`
(Linux 5.1、2019 年) はこの考えをさらに
進めた：カーネルと共有するキュー。

誰が誰を待つ？



クライアント 32、まず 1 つ、次に 8 つが遅い (1 秒)

実際の出力

```
$ ./http_server -m iterative & ./load PORT 1 31 1000;  
./load PORT 8 24 1000  
32 replies 200 OK of 32  
slow 1: median 1001 ms, max 1001 ms  
fast 31: median 981 ms, max 982 ms  
32 replies 200 OK of 32  
slow 8: median 1001 ms, max 1001 ms  
fast 24: median 982 ms, max 983 ms
```

実際の出力

```
$ ./http_server -m epoll & ./load PORT 1 31 1000; ./load  
PORT 8 24 1000  
32 replies 200 OK of 32  
slow 1: median 1001 ms, max 1001 ms  
fast 31: median 1 ms, max 2 ms  
32 replies 200 OK of 32  
slow 8: median 1001 ms, max 1001 ms  
fast 24: median 1 ms, max 2 ms
```

反復型サーバは遅いクライアント 1 つのために全員を待たせるが、スレッド 1 つの `epoll` は待たせない。8 スレッドのプールは遅いクライアント 1 つなら持ちこたえるが、8 つでは無理 (A8.2)。

まとめ

- ソケットはディスクリプタ：ファイルやパイプと同じく `read`、`write`、`close`
- TCP：信頼できるストリームで、接続はカーネルが確立する。UDP：保証のないデータグラム
- アドレスはネットワークバイト順（`htons`）で、`getaddrinfo` で解決する
- 誰が待つかはサーバモデルで決まる：プロセス、スレッド、プール、イベント

解答付き TD7 + AI チャレンジ

- `sock`、`echo_server`（反復型と `fork` 版）
- 5 つのモデルを持つ `http_server`
- 新規： `tcp_states`、`backlog`、`load`、`udp`、`addr`、`local`

Bash

```
make -C tds && tds/tests/run.sh td7
```



Universidad
Indoamérica

まとめ

3つの問い

1. なぜ `connect` は、サーバが `accept` を呼ぶ前に終わってしまうのか？
2. あなたのノートパソコンで `sin_port = 7000` はどのポートを開くか、それはなぜか？
3. スレッド1つで遅いクライアント 8 つに耐えられるサーバモデルはどれか？